

# YouTube View Predictor Machine Learning Project

Aly Camara, Maya M. Laidler

## Abstract

Ridge Linear Regression and K-Nearest Neighbors (KNN) machine learning approaches were used to predict the number of views a YouTube video would receive based on video metadata. A Convolutional Neural Network (CNN) was also used to classify thumbnail images and predict view counts. We progressively improved our models by adding Term Frequency-Inverse Document Frequency (TF-IDF) text features from video titles and sentiment analysis features. We found that Ridge regression consistently outperformed KNN, and that model performance improved dramatically when trained on a larger dataset. Our best model, Ridge regression with TF-IDF and sentiment features trained on the Kaggle YouTube trending dataset, achieved an RMSE of 0.62 on the log-transformed view count. We then discuss how these models may inadvertently reinforce platform biases by learning to predict success based on patterns that already favor established creators with established socio-political norms.

## 1 Introduction

YouTube is the largest video sharing platform in the world, with billions of views generated daily. As the platform has grown into a major source of income for content creators and a key channel for advertisers, understanding what drives a video's popularity has become increasingly valuable. Predicting how many views a video will receive based on its metadata is a well-motivated machine learning problem with real applications for both creators and platforms. Prior work has explored this area using a variety of methods. Niture (2021) applied linear regression and several classification models to YouTube trending video data, finding that features such as likes, comment counts, and publish time are meaningfully correlated with view counts, and that a log-transformed linear regression model can achieve moderate predictive accuracy.

This project seeks to build a predictive model for the number of views a YouTube video will receive based on features available at the time of upload, such as the video title, publish time, video category, and thumbnail. We deliberately exclude post-upload features such as likes, dislikes, and comment counts, as a content creator would not have access to these before posting.

This project is motivated by a practical gap in the existing research: most prior work on YouTube view prediction either uses post-upload engagement features that creators cannot control, or focuses on identifying trending videos after the fact. We are interested in whether a model trained only on features known at upload time can meaningfully predict a video's eventual reach. For content creators, such a model could inform decisions about when to post, how to title a video, and what kind of thumbnail to use. For platform researchers, it raises the broader question of how much of a video's popularity is determined before anyone has even watched it.

Our research questions are:

1. Can we accurately predict the number of views a YouTube video will receive using only metadata features available at the time of upload?
2. How does model performance compare across Ridge regression, K-Nearest Neighbors, and a Convolutional Neural Network trained on thumbnail images?

## 2 Data

### 2.1 Scraped Dataset

We collected our own dataset using the YouTube Data API v3. We wrote a function that searches YouTube by video category and retrieves metadata for each video. We looped through all available YouTube categories and collected approximately 1,605 videos, saving the results as a CSV file.

The scraped dataset contains the following variables: **video\_id** (string, unique identifier), **title** (string, video title), **published\_at** (datetime, upload timestamp), **channel\_id** (string, unique channel identifier), **views** (integer, total view count, range: 0 to 16.8 billion, heavily right-skewed), **likes** (integer, total like count), **comments** (integer, total comment count), **duration** (string, ISO 8601 format, converted to seconds for modeling), and **category\_id** (integer, YouTube video category, 1-28).

We derived several additional features: **title\_length** (integer, character count of title), **publish\_hour** (integer, 0-23), **publish\_dayofweek** (integer, 0-6), **publish\_month** (integer, 1-12), **is\_weekend** (binary, 1 if Saturday or Sunday), **channel\_video\_count** (integer, number of videos in the dataset from the same channel), and **channel\_avg\_views** (float, average views across a channel’s videos in the dataset).

Several features required additional processing. The duration field was returned as a raw ISO 8601 string (e.g. "PT4M13S") rather than a number, so we wrote a parsing loop to convert it into total seconds. The title and channel ID fields are strings that Ridge regression cannot handle directly, so we used title length as a numeric proxy for title content and excluded channel ID from the feature set initially.

We log-transformed the view count before training to account for the heavily right-skewed distribution of views, where a small number of videos accumulate the vast majority of views.

## 2.2 Kaggle Dataset

Following feedback suggesting we look for a larger existing dataset, we also obtained the YouTube Trending Video Statistics dataset from Kaggle (2), which was also used by Niture (2021). This dataset contains 40,949 rows of US trending video metadata spanning 2017 to 2018. After removing duplicate videos that appeared on the trending list on multiple days, we retained 25,136 unique videos. This dataset does not include video duration or channel-level statistics, but contains the same core features: title, publish time, category ID, views, likes, comment count, and thumbnail URL. We applied the same feature derivation pipeline as the scraped dataset and the same log transformation to the view count.

## 3 Methods

### 3.1 Feature Engineering

For both datasets, our base numeric feature set consisted of: title length, publish hour, is\_weekend, and category ID. For the scraped dataset we additionally included duration in seconds, channel video count, and channel average views.

We progressively added two additional feature types:

**TF-IDF Title Features:** We used `TfidfVectorizer` from scikit-learn to convert video titles into numeric features. TF-IDF assigns higher weights to words that are distinctive to a particular video’s title relative to all titles in the dataset. This allowed us to capture the actual content of titles rather than just their length.

**Sentiment Features:** Using the VADER sentiment analyzer (2), we computed three sentiment scores for each video title: positivity, negativity, and neutrality. VADER is specifically tuned for short social media text, making it well-suited for YouTube titles.

We used a `ColumnTransformer` to combine numeric and text features into a single pipeline, applying `StandardScaler` to numeric features and `TfidfVectorizer` to the title text simultaneously.

### 3.2 Ridge Regression

Ridge regression is a linear regression model that introduces an L2 regularization penalty term to prevent overfitting:

$$RSS_{L2} = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 + \lambda \sum_{j=1}^P \beta_j^2 \quad (1)$$

The L2 penalty discourages any single variable from dominating, spreads importance across correlated features, and stabilizes the solution when features are highly correlated. The strength of the regularization is controlled by the hyperparameter  $\alpha$  (equivalent to  $\lambda$ ). We used 10-fold cross-validation with `KFold` to select the best  $\alpha$  from the set  $\{0.01, 0.1, 1.0, 10.0, 100.0\}$ .

### 3.3 K-Nearest Neighbors

K-Nearest Neighbors (KNN) regression predicts the value of a new observation by averaging the values of its  $k$  closest neighbors in the feature space. Unlike Ridge, KNN is a non-parametric method that makes no assumptions about the

functional form of the relationship between features and the target. We used 10-fold cross-validation to select the best  $k$  from 1 to 100 on the scraped dataset. On the larger Kaggle dataset, we sampled a smaller set of  $k$  values due to the computational cost of KNN on large datasets, as KNN must compute distances to every point in the training set for each prediction, making it slow at scale.

### 3.4 Convolutional Neural Network

A Convolutional Neural Network (CNN) was used to classify thumbnail images and predict view counts. The criterion used was Mean Squared Loss and the optimizer used was Adam. The CNN takes in a thumbnail image and combines the extracted image features with additional numeric metadata features. The architecture of the combined model is as follows:

Input features  $\rightarrow$  Linear  $\rightarrow$  BatchNorm  $\rightarrow$   
ReLU  $\rightarrow$  Dropout  $\rightarrow$  Linear  $\rightarrow$  ReLU

The output of the CNN is combined with the additional metadata features to make a final prediction.

### 3.5 Cross-Validation

For all models, we used 10-fold cross-validation (`KFold`, `n_splits=10`, `shuffle=True`) to evaluate model performance and tune hyperparameters. Cross-validation splits the training data into 10 folds, trains on 9 and validates on 1, rotating through all folds and averaging the results. This gives a more reliable performance estimate than a single train/test split.

## 4 Results

Our results show a clear progression of improvement as we added features and increased dataset size. Starting from a basic Ridge model with numeric features only (RMSE 3.55), we saw meaningful improvements from adding TF-IDF title features (RMSE 2.97) and further improvement from adding sentiment features with proper hyperparameter retuning (RMSE 2.62). The most dramatic improvement came from training on the larger Kaggle dataset, where our best model achieved an RMSE of 0.62, a reduction of over 75% from the baseline.

Ridge regression consistently outperformed KNN across all feature sets and both datasets. On the Kaggle dataset, KNN's best  $k$  was 2, and RMSE increased steadily as  $k$  grew larger, suggesting overfitting. This is likely due to

the curse of dimensionality, as KNN struggles in high-dimensional spaces and the TF-IDF features add many dimensions to the feature space. Ridge handles high-dimensional data more gracefully due to its regularization.

An important methodological finding was that adding new features requires retuning hyperparameters. When we added sentiment features without retuning, Ridge performance was unchanged (RMSE 2.97). After retuning, the best  $\alpha$  shifted from 10.0 to 1.0 and RMSE improved to 2.62.

| Model                                    | Data | RMSE        |
|------------------------------------------|------|-------------|
| Ridge (basic)                            | S    | 3.55        |
| Ridge CV ( $\alpha=10.0$ )               | S    | 3.71        |
| KNN CV ( $k=12$ )                        | S    | 3.57        |
| Ridge + TF-IDF                           | S    | 2.97        |
| KNN + TF-IDF                             | S    | 3.30        |
| Ridge + TF-IDF + Sent. ( $\alpha=10.0$ ) | S    | 2.97        |
| KNN + TF-IDF + Sent. ( $k=7$ )           | S    | 3.47        |
| Ridge + TF-IDF + Sent. ( $\alpha=1.0$ )  | S    | 2.62        |
| Ridge + TF-IDF + Sent. ( $\alpha=0.1$ )  | K    | <b>0.62</b> |
| KNN + TF-IDF + Sent. ( $k=2$ )           | K    | 0.74        |

Table 1: Model performance. S = Scrapped dataset, K = Kaggle dataset.

| $k$ | RMSE   |
|-----|--------|
| 1   | 0.7118 |
| 3   | 0.7486 |
| 5   | 0.8995 |
| 7   | 1.0613 |
| 10  | 1.2401 |
| 15  | 1.3854 |
| 20  | 1.4549 |
| 30  | 1.5226 |
| 50  | 1.5827 |

Table 2: RMSE scores for different values of  $k$  on the Kaggle dataset. The best performing parameter was  $k = 1$ .

#### 4.1 CNN Results

The initial CNN model produced results that were barely better than random guessing (RMSE: 1.7256,  $R^2$ : 0.0199). After including additional features and subscriber count data retrieved via the YouTube API, performance improved (RMSE: 1.5874,  $R^2$ : 0.1706). Due to API quota limits, subscriber count data was only available for approximately 5,000 videos, which became the final training dataset size for the CNN.

After normalizing views by subscriber count and adding additional features, the CNN achieved its best results (RMSE: 0.6772,  $R^2$ : 0.6535). Table 3 shows the training and validation metrics across epochs, with early stopping triggered at epoch 28.

Model generalization was evaluated using the generalization gap metric (validation loss minus training loss), where positive values indicate overfitting and negative values suggest underfitting. The average gap of -0.036 indicates the model achieved good generalization.

Figure 1 represents the data shown in Table

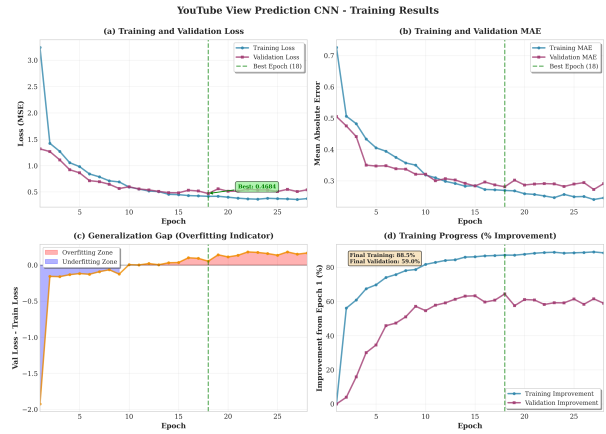


Figure 1: Training results for the CNN

3 visually. It shows (a) training and validation loss curves over 28 epochs, with the best model (lowest validation loss of 0.4684) achieved at epoch 18, (b) mean absolute error metrics demonstrating consistent improvement in both training and validation sets, (c) the generalization gap highlighting the absence of overfitting as evidenced by minimal divergence between training and validation performance, and cumulative improvement showing 87.2 percent and 64.4 percent reduction in training and validation loss respectively from the initial epoch.

| Epoch | Train Loss | Train MAE | Val Loss | Val MAE |
|-------|------------|-----------|----------|---------|
| 1     | 3.2423     | 0.7261    | 1.3169   | 0.5054  |
| 2     | 1.4222     | 0.5065    | 1.2652   | 0.4753  |
| 3     | 1.2685     | 0.4825    | 1.1078   | 0.4413  |
| 4     | 1.0543     | 0.4335    | 0.9213   | 0.3501  |
| 5     | 0.9797     | 0.4056    | 0.8618   | 0.3473  |
| 6     | 0.8411     | 0.3945    | 0.7134   | 0.3481  |
| 7     | 0.7850     | 0.3750    | 0.6933   | 0.3384  |
| 8     | 0.7098     | 0.3572    | 0.6447   | 0.3374  |
| 9     | 0.6890     | 0.3501    | 0.5638   | 0.3209  |
| 10    | 0.5921     | 0.3186    | 0.5970   | 0.3209  |
| 11    | 0.5541     | 0.3098    | 0.5553   | 0.3002  |
| 12    | 0.5168     | 0.2984    | 0.5373   | 0.3067  |
| 13    | 0.5051     | 0.2915    | 0.5091   | 0.3028  |
| 14    | 0.4537     | 0.2830    | 0.4853   | 0.2920  |
| 15    | 0.4478     | 0.2845    | 0.4826   | 0.2836  |
| 16    | 0.4300     | 0.2724    | 0.5310   | 0.2962  |
| 17    | 0.4241     | 0.2714    | 0.5167   | 0.2870  |
| 18    | 0.4154     | 0.2693    | 0.4684   | 0.2810  |
| 19    | 0.4170     | 0.2679    | 0.5590   | 0.3020  |
| 20    | 0.3992     | 0.2588    | 0.5117   | 0.2867  |
| 21    | 0.3804     | 0.2566    | 0.5149   | 0.2898  |
| 22    | 0.3676     | 0.2517    | 0.5495   | 0.2911  |
| 23    | 0.3615     | 0.2463    | 0.5363   | 0.2902  |
| 24    | 0.3780     | 0.2566    | 0.5372   | 0.2820  |
| 25    | 0.3716     | 0.2491    | 0.5077   | 0.2896  |
| 26    | 0.3664     | 0.2499    | 0.5482   | 0.2943  |
| 27    | 0.3553     | 0.2403    | 0.5059   | 0.2723  |
| 28    | 0.3731     | 0.2457    | 0.5405   | 0.2912  |

Table 3: Training and validation metrics per epoch (early stopping at epoch 28).

## 5 Discussion

Our results partially answer our first research question: metadata features available at upload time do carry predictive signal for view counts, but the relationship is complex and noisy. The improvement from RMSE 3.55 to 0.62 across our experiments shows that the choice of features and dataset size matter enormously, but even our best model leaves substantial variance unexplained. This aligns with Niture (2021), who found that a log-transformed linear regression model achieves moderate accuracy on similar data, and suggests that a meaningful portion of a video’s eventual popularity may be driven by factors that are difficult or impossible to capture from metadata alone, such as timing relative to trending topics, algorithmic promotion, or social sharing.

Our second research question produced a clear answer: Ridge regression outperformed KNN consistently, and the CNN achieved com-

petitive results after normalizing for subscriber count. The superiority of Ridge over KNN is consistent with the theoretical expectation that distance-based methods struggle in high-dimensional spaces, and the TF-IDF features introduce many dimensions. The CNN’s strong performance after subscriber count normalization suggests that thumbnail content is genuinely informative, but that raw view counts are dominated by channel size in ways that obscure the signal from visual features.

### 5.1 Limitations

Several limitations of this work are worth noting. First, both datasets are biased toward popular or trending videos. Our scraped dataset was collected by searching category names, which tends to surface higher-performing content, and the Kaggle dataset contains only videos that reached YouTube’s trending page. This means our model learns what distinguishes highly viewed videos from other highly viewed videos, not what separates viral content from obscure content. Predictions for newly uploaded videos from small channels may therefore be unreliable. Second, the Kaggle dataset spans only 2017 to 2018, and YouTube’s content landscape, algorithm, and creator demographics have changed substantially since then. Models trained on this data may not generalize well to current videos. Third, our scraped dataset of 1,605 videos is relatively small, and the improvements we observed when switching to the 25,136-video Kaggle dataset suggest that more data would further improve performance.

## 6 Sociopolitical Analysis

YouTube’s recommendation and trending algorithms have significant real-world consequences for whose content gets amplified. A model that predicts views based on metadata may inadvertently encode existing platform biases. For example, if videos from certain categories, languages, or creator demographics have historically received more algorithmic promotion, a model trained on this data will learn to predict high views for those same patterns. This means a creator using such a model to optimize their content could be nudged toward producing videos that conform to whatever the platform already rewards, reinforcing existing inequalities in visibility and income. Additionally, the Kaggle dataset used in this project contains only trending videos, which introduces survivorship bias. Our model learns

what makes already-popular videos popular, not what gives lesser-known creators a chance to break through.

In addition, because our model was learned on already popular videos, it may overrepresent non-marginalized groups of people. It may underrepresent content from marginalized creators, non-English languages, and niche communities. This means that our model’s predictions may systematically undervalue diverse perspectives while overvaluing content that aligns with dominant cultural and linguistic norms already favored by the platform. These norms may include exaggerated, male, white, and English-speaking content. Furthermore, by learning to predict views primarily from thumbnail images, our model may incentivize clickbait visual strategies such as exaggerated faces and extreme views. This contributes to the platform’s broader trend toward pushing viewers to extreme views and ideas.

## 7 Conclusions

Our results demonstrate that predicting YouTube view counts from video metadata is a tractable machine learning problem, and that performance improves substantially with better features and more data. Adding title text via TF-IDF was the single most impactful improvement, suggesting that what a video is called matters more than simple numeric metadata. Sentiment features provided additional gains, though only when hyperparameters were retuned after adding them. Ridge regression outperformed KNN throughout, likely because the high-dimensional TF-IDF feature space disadvantages distance-based methods. Training on the larger Kaggle dataset reduced RMSE from 2.62 to 0.62, highlighting how much dataset size matters. The CNN achieved an  $R^2$  of 0.65 after normalizing by subscriber count, suggesting thumbnail content carries meaningful predictive signal beyond metadata alone.

### 7.1 What We Would Do Differently

If we were starting this project over, we would prioritize data collection earlier and more systematically. We hit API quota limits repeatedly during development, which slowed our progress and limited the size of our scraped dataset significantly. We would set up multiple API keys in advance and design the data collection loop to run in smaller batches over multiple days. We would also collect subscriber count data from

the start, since adding it to the CNN improved performance substantially but required a separate round of API calls that we were only partially able to complete. More broadly, we would spend more time on exploratory data analysis before modeling, including visualizations of the view count distribution by category, time of day, and title sentiment, which would have helped us make better-informed decisions about feature engineering earlier in the process.

## References

- Niture, A.A. (2021). Predictive analysis of YouTube trending videos using Machine Learning. *MSc Dissertation*, Dublin Business School.
- J, Mitchell. (2019). YouTube Trending Video Statistics. Retrieved from <https://www.kaggle.com/datasets/datasnaek/youtube-new>
- Hutto, C.J. and Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*.